AGENDA

Scope of This Review

- Requirements (Functional & Non-Functional)
- Detailed System Architecture
- Use Case & ER Diagrams
- Module Description & Tech Stack
- UI Design, Frontend & Backend Build
- Database & API Integration
- Progress (70%), Screenshots & Challenges

01

REQUIREMENTS

Functional & Non-Functional Requirements

Functional Requirements

- User registration & authentication
- Role-based dashboards
- Core feature: [add/search/manage records]
- CRUD operations via REST APIs
- Search, filter & sort functionality
- Notifications / status updates

Non-Functional Requirements

- Performance: response < 2 seconds
- Scalability: handle concurrent users
- Security: JWT auth, password hashing
- Usability: responsive, mobile-friendly UI
- Availability & reliable data storage
- Maintainability: modular MERN codebase

02

ARCHITECTURE

System Architecture (Detailed)



HTTP / HTTPS requests flow from the client, through Express REST endpoints, to MongoDB — with JSON responses returned along the same path.

Update this slide with your own architecture diagram (draw.io / Lucidchart export) showing your project's specific components, external APIs, and deployment setup.

03

USE CASE

Use Case Diagram

Insert your Use Case Diagram
(actors ↔ system use cases)

Include

- Identify all actors (Admin, User, etc.)
- List each actor's use cases
- Show include / extend relationships
- Keep it to system-level interactions
- Export as image and paste here

04

ER DIAGRAM

ER Diagram / Database Design

Insert your ER Diagram
(entities, attributes & relationships)

Include

- List entities and attributes
- Mark primary & foreign keys
- Show relationships (1:1, 1:N, N:N)
- Reflect actual MongoDB collections
- Note any embedded vs. referenced docs

05

MODULES

Module Description

Module	Description	Key Functions
User Management	Registration, login & role handling	Signup, Login, JWT Auth
Core Module 1	[Project-specific main feature]	Create, View, Update
Core Module 2	[Project-specific supporting feature]	Search, Filter, Sort
Admin Module	Manage users & records	Approve, Delete, Reports
Notification Module	Alerts & status updates	Email / In-app alerts

06

TECH STACK

Technology Stack

MongoDB

NoSQL Database

Express.js

Backend Framework

React.js

Frontend Library

Node.js

Runtime Environment

Supporting Tools

- Mongoose (ODM)
- JWT & bcrypt (auth/security)
- Postman (API testing)
- Git & GitHub (version control)
- VS Code, npm
- Axios (HTTP client)

07

UI DESIGN

UI Design / Wireframes

Insert wireframes / mockups for key application screens

Include

- Show key screen wireframes
- Login/Signup, Dashboard, Core screens
- Note responsive breakpoints
- Mention design tool used (Figma, etc.)
- Replace with real wireframe images

08

FRONTEND

Frontend Development

- Built with React.js using functional components & hooks
- Component-based structure: Navbar, Forms, Cards, Tables, Modals
- State management via Context API / Redux
- Client-side routing implemented with React Router
- Axios used to consume backend REST APIs
- Responsive UI styled with [Tailwind / Bootstrap / CSS Modules]

Tip: list the actual pages/components completed so far, e.g. Login, Dashboard, Profile, Item List.

09

BACKEND

Backend Development

- Node.js + Express.js server with modular route structure
- MVC pattern: separate routes, controllers & models
- Authentication middleware using JWT
- Input validation & centralized error handling
- Environment-based configuration (.env, dotenv)
- Server tested locally with Postman before integration

Tip: mention specific middleware, validation library, or error-handling strategy you used.

DATABASE

Database Integration

- MongoDB Atlas / local MongoDB used as the data store
- Mongoose schemas defined for each collection
- Relationships modeled via referencing / embedding
- Indexes added for frequently queried fields
- CRUD operations connected end-to-end with the backend
- Sample data seeded for testing during development

Tip: name your actual collections (e.g. Users, Products, Orders) here.

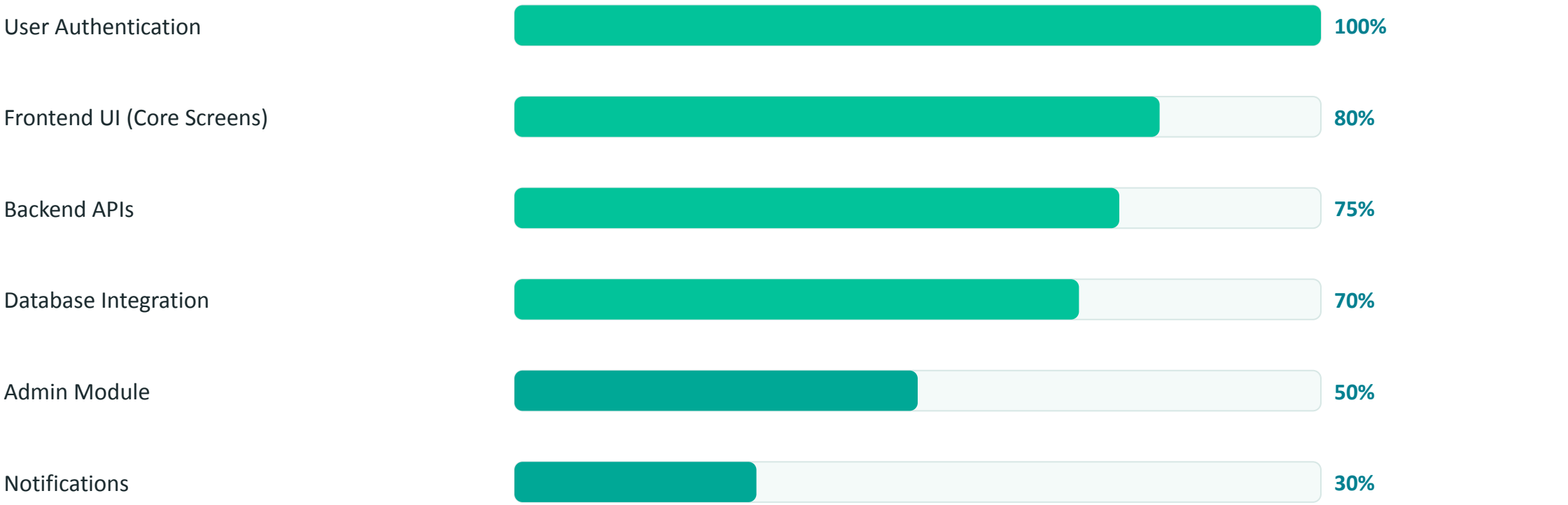
API

API Development

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	Authenticate & issue JWT
GET	/api/items	Fetch all records
POST	/api/items	Create a new record
PUT	/api/items/:id	Update an existing record
DELETE	/api/items/:id	Delete a record

STATUS

Module-wise Implementation Status



SCREENSHOTS

Screenshots

[Login / Signup Page screenshot]

[Dashboard screenshot]

[Core Feature Screen screenshot]

[Admin Panel screenshot]

CHALLENGES

Challenges Faced & Solutions

Challenge	Solution Adopted
CORS errors during API integration	Configured CORS middleware in Express
JWT token expiry handling	Implemented refresh-token / re-login flow
State sync between components	Adopted Context API for shared state
Slow query performance	Added MongoDB indexes on key fields

NEXT STEPS

Remaining Work & Plan

- Complete Admin Module: analytics dashboard & reports
- Implement Notification Module (email / in-app alerts)
- Conduct integration & user-acceptance testing
- Improve UI polish, accessibility & responsiveness
- Deploy application (Render/Vercel/Netlify + MongoDB Atlas)
- Prepare final documentation & demo for final review

NEXT STEPS

References

#	Reference
1	MongoDB, Express, React, Node.js — Official Documentation
2	IEEE / journal papers cited in the literature survey (Review 0)
3	Relevant textbooks / course material used
4	Any third-party APIs or libraries used in the project